

Self-Improving Systems and Media Quines

An Admissibility-Theoretic Approach to Recursive Generation

Flyxion
Independent Researcher

2026

Abstract

This monograph develops a theory of self-improving computational systems and the media artifacts they produce, grounded in an admissibility-theoretic account of distinction, repair, and continuation. Ordinary recursive self-modification is shown to be underspecified as an engineering goal: without an admissibility constraint governing which successor states are reachable and diagnosable, “self-improvement” collapses into unconstrained drift indistinguishable from decay. We introduce the notion of a *media quine*: an artifact whose successful interpretation reconstructs, in the observer, the generative operator responsible for the artifact’s own production, rather than merely reproducing its surface content. Media quines are shown to be the communicative analogue of admissible reconstruction, and to supply the missing constraint that separates genuine self-improvement from mere self-modification. Using the Spherepop calculus (Pop, Refuse, Bind, Collapse) as a primitive operator alphabet, and Chain of Memory as the fixed-point criterion for successful transmission, we sketch an architecture in which successive generations of a self-improving system stand in a media-quine relation to one another: each generation is not merely produced by its predecessor but reconstructs, and can therefore inherit, the constraint structure that produced it. The monograph closes by identifying the principal failure modes of such systems — irrecoverable drift, non-admissible mutation, and loss of the Chain-of-Memory fixed point — and situates the resulting picture within the broader operator-ecology account of persistence.

Dependency Table

The concepts developed in this monograph are cumulative rather than parallel: each later notion is defined in terms of specific earlier ones, and the central technical result of Chapter 5 is the point at which most of the preceding machinery converges. The table below is offered as a map of that structure, to be consulted rather than read straight through.

Concept	Depends on	Used by
Distinction (§1.1)	—	Admissibility, Operator Ecology
Admissibility (§1.2)	Distinction	Repair/Reconstruction, Media Quine, Reconstructible Lineage
Repair / Reconstruction (§1.3)	Admissibility	Chain of Memory, Reconstruction Chapter 5
Chain of Memory (§1.4)	Reconstruction	Media Quine
Operator Ecology (§1.5)	Distinction	Media Quine as Operator Factory (Ch. 4), Failure Modes (Ch. 7)
Continuation-Preserving Recursion (§2.3)	Admissibility, Repair	Media Quine, Self-Modification Lineages (§5.2)
Media Quine, Def. 3.1	Chain of Memory, Admissibility	Reconstructible Lineage, Architecture (Ch. 6)
Spherepop Primitives (§4.2)	—	Media Quine as Operator Factory, Architecture (Ch. 6)
Reconstructible Lineage, Def. 5.1	Media Quine	Continuation Criterion (Prop. 5.2)
Continuation Criterion, Prop. 5.2	Reconstructible Lineage, Operator Ecology	Architecture (Ch. 6), Failure Modes (Ch. 7)
Architecture (Ch. 6)	Continuation Criterion, Spherepop Primitives	Failure Modes (Ch. 7)

The Continuation Criterion (Proposition 5.2, Section 5.5) is the point every earlier chapter builds toward and every later chapter depends on; it is the monograph’s principal technical result, and the remaining chapters are best read as, respectively, its constructive realization (Chapter 6) and its limits (Chapters 7–8).

Contents

1	Foundations	1
1.1	Distinction Before Object	1
1.2	Admissibility	1
1.3	Identity Versus Continuation	2
1.4	Repair and Reconstruction	2
1.5	Chain of Memory	2
1.6	Operator Ecology	3
1.7	RSVP as Physical Substrate	3
2	Self-Improving Code as Continuation Geometry	5
2.1	Why “Self-Improving” Is Underspecified	5
2.2	Continuation as Operator Composition	5
2.3	Recursion Without Amnesia	6
2.4	Why Fitness Is Insufficient	6
2.5	The Bridge to Media Quines	6
3	Quines Generalized	9
3.1	The Classical Quine	9
3.2	From Source-Code Self-Reproduction to Media Quines	9
3.3	Media Quines as Generalized Fixed Points	10
3.4	The Three-Level Hierarchy	10
3.5	Partial Media Quines	11
3.6	Admissibility, Not Fidelity	11
3.7	Archives as Computational Substrates	11
3.8	Compression and Latent Operators	12
4	The Media Quine as Operator Factory	13
4.1	Reading as Installation	13
4.2	Interpretation as Execution	13
4.3	Spherepop as a Primitive Alphabet	14
4.4	Minimal Operator Bases	14
4.5	Selection, Exclusion, Coupling, Projection in the Interpreter	14
4.6	Operator Factories and Operator Ecology	15

5	Reconstruction Under Recursive Self-Modification	17
5.1	The Moving-Target Problem	17
5.2	Self-Modification Lineages	17
5.3	Local Versus Global Reconstruction	18
5.4	Drift, Repair, and the Cost of Amnesia	18
5.5	Repair Operators Must Themselves Survive	19
5.6	The Continuation Criterion	19
5.7	Necessary Versus Sufficient Conditions	20
6	Building the Thing	21
6.1	Design Requirements Restated	21
6.2	A Generation as a Spherepop History	21
6.3	The Modification Operator as Bind–Collapse	21
6.4	The Verification Operator as Pop–Refuse	22
6.5	Trust Boundaries	22
6.6	A Worked Toy Lineage	23
6.7	Externalized Chain of Memory	24
6.8	Where This Meets Operator Ecology in Practice	24
7	Failure Modes and Limits	25
7.1	Formal Satisfaction Without Practical Reconstructibility	25
7.2	Non-Admissible Mutation Passing as Admissible	25
7.3	Gradual Catastrophe	26
7.4	Loss of the Chain-of-Memory Fixed Point	26
7.5	Goodhart’s Law and Recursive Improvement	26
7.6	The Cost of Maintaining Full History	27
7.7	Scope of the Claims	27
8	Civilizational and Cognitive Implications	29
8.1	Beyond the Single Lineage	29
8.2	Keystone Operators Revisited	29
8.3	Cognition as a Self-Modification Lineage	30
8.4	Media Quines Across Domains	30
8.5	Closing Remark	31
A	Admissibility Spaces and Continuation Geometry	33
B	Media Quines as Fixed Points	35
C	Reconstruction Metrics	37
D	Category-Theoretic Formulation	39
E	Spherepop Algebra	41
F	Dynamical Systems	43
G	Information Without Representation	45

Chapter 1

Foundations

1.1 Distinction Before Object

The argument of this monograph presupposes a specific ordering among concepts that are usually treated as co-primitive, or in the opposite order. Standard treatments of computation begin with objects: states, data structures, programs as fixed artifacts, and ask afterward how those objects change. The framework used here begins instead with distinction: a system is characterized first by which differences it can register and preserve, and only derivatively by the objects those distinctions are taken to individuate. An object, on this view, is a stabilized pattern of distinctions that has survived enough repair and reconstruction to be treated as a persistent thing. This is not a merely terminological preference. It has a direct consequence for self-improving systems: a system cannot improve what it cannot distinguish, and it cannot preserve an improvement unless the distinction responsible for that improvement is itself reconstructible after the system has changed.

1.2 Admissibility

Call the set of states reachable from a given configuration, under the operators available to a system, that configuration's *admissibility volume*. Admissibility is not a claim about what a system currently is, but about what it can become without passing through a state from which its defining distinctions can no longer be recovered. A self-improving system is therefore not usefully defined by the direction of its modifications — whether some scalar fitness measure increases — but by whether successive states remain inside a shared admissibility volume: whether an observer, given a later state, could in principle reconstruct enough of the earlier constraint structure to certify that the change was a modification of the same system rather than its replacement by an unrelated one.

This distinction matters because unconstrained recursive self-modification and genuine self-improvement are observationally similar in the short run and radically different in the long run. Both produce a sequence of distinct program states. They differ in whether that sequence remains admissibly connected: whether later states carry enough diagnostic structure to be repaired, audited, or reconstructed in light of earlier ones. A system that merely accumulates modification without preserving admissibility is not self-improving in any sense stronger than drift; the appearance of improvement is a property of the fitness function used to describe it from outside, not a property intrinsic to the system's own trajectory.

1.3 Identity Versus Continuation

It is worth forestalling a natural misreading before proceeding. Nothing in the admissibility criterion above claims that a system modified over time remains numerically identical to its earlier self, in the sense debated in the philosophical literature on personal identity and object persistence. That literature typically asks what makes a later thing the same thing as an earlier one, and treats sameness as a relation that either holds or fails to hold, often turning on criteria — spatiotemporal continuity, psychological continuity, sortal essentialism — that are largely orthogonal to the concerns of this monograph. Admissibility asks a narrower and more tractable question: not whether M_{i+1} is the same system as M_i , but whether M_{i+1} remains reconstructibly connected to M_i by operators that preserve enough diagnostic structure to repair, audit, or continue the trajectory. A continuation is a claim about recoverability, not a claim about metaphysical sameness, and the two can come apart in either direction: two systems can be admissibly continuous while differing in every component (nothing here requires persistence of parts, only persistence of reconstructible structure), and two systems can be materially identical at some instant while standing in no admissible relation to one another, if the process that produced that instantaneous match discarded the operators needed to reconstruct it. Readers arriving from debates about ship-of-Theseus-style identity should treat continuation as a deliberately weaker and more operational notion, chosen because it is what self-improving systems actually need and can be checked for, not because it settles or even engages the metaphysical question.

1.4 Repair and Reconstruction

Repair theory treats correctness as secondary to diagnosability: a system is well-behaved not because it avoids all faults, but because its faults remain locatable and correctable using the structure the system itself retains. Reconstruction generalizes this from single faults to entire histories: given a degraded or transformed state, what invariants of the original constraint structure can still be recovered, and what has been irrecoverably lost. Self-improving systems are reconstruction problems running forward in time rather than backward: each new generation is a transformation of the previous one, and the question of whether the system is actually improving is, formally, the question of how much of the prior generation's constraint structure the new generation can reconstruct, extend, or repair, as opposed to how much it has silently discarded.

1.5 Chain of Memory

Chain of Memory (CoM) names the criterion by which a semantic or computational state counts as successfully transmitted rather than merely copied: transmission succeeds when the receiving system arrives at a state from which the same operator that produced the original state can itself be regenerated. A CoM fixed point is a state that reproduces, in whoever or whatever receives it, the operator responsible for its own production, so that the transmission chain does not require the original source to persist. This is the criterion that will be generalized, in Chapter 3, into the definition of a media quine: a media quine is precisely an artifact engineered to be a Chain of Memory fixed point under interpretation.

1.6 Operator Ecology

Individual operators — a repair procedure, a generative rule, a self-modification routine — do not persist in isolation. They persist because a network of enabling constraints keeps them executable: prerequisite operators, maintained infrastructure, available training or documentation, and so on. This is the operator-ecology picture: what is at stake in the persistence of a system is not the survival of any particular state or even any particular operator, but the survival of the ecology of constraints that keeps a repertoire of operators executable. Self-improving systems will be shown, in later chapters, to be a special case in which the system's own operator ecology is itself a target of modification — the system does not merely act on the world with a fixed repertoire, it acts on its own repertoire, which is precisely what makes the admissibility question non-trivial: an operator ecology can improve its own reach, or it can quietly destroy the operators it would need to recover from a bad modification, and from the outside these two trajectories can look identical for a long time.

1.7 RSVP as Physical Substrate

Where the preceding sections describe the theory in operator- and information-theoretic terms, RSVP (scalar field Φ , vector field v , entropy field S) supplies the physical register in which any concrete implementation of a self-improving system must actually run: computation is not a disembodied abstraction but a process embedded in a substrate with its own entropy production and dissipation structure. This monograph does not depend on RSVP's cosmological content, but it inherits from RSVP the general commitment that admissibility and repair are ultimately constraints on physically realizable trajectories, not merely constraints on formal descriptions of them. A self-improving system that appears admissible at the level of its source code but requires physically unrealizable repair operators to recover from its own modifications is not, in the sense used here, actually admissible.

Chapter 2

Self-Improving Code as Continuation Geometry

2.1 Why “Self-Improving” Is Underspecified

Most informal discussions of self-improving code specify only a direction: the system should get better, faster, more capable, or more efficient by some external measure, and it should do so by modifying its own code or weights rather than being modified by an external designer. This specification is silent on the question that matters most for the present framework, namely what is supposed to persist across the modification. A system that discards its own diagnostic structure at every step can satisfy an external fitness measure while becoming, from the inside, unrepairable: each successive version is a different system wearing the fitness score of the one before it. Continuation geometry supplies the missing half of the specification: self-improvement is modification that remains inside a continuation — a sequence of states connected by operators that preserve enough of the prior state’s distinguishing structure that the sequence can be treated, and repaired, as a single evolving system rather than a series of unrelated replacements.

2.2 Continuation as Operator Composition

A continuation, in the sense used throughout this program, is not merely a sequence of states but a sequence together with the operators that connect them and the diagnostic coordinates that remain legible at each step. Self-modifying code is naturally described as a composition of operators acting on a representation that includes, at minimum, the code itself, whatever state the code manipulates, and a record — explicit or reconstructible — of the history that produced the current code. The central formal claim of this chapter is that treating self-improvement as operator composition rather than as state replacement is what makes admissibility checking possible at all: one can ask, of an operator, whether its image remains inside the admissibility volume of its domain; one cannot meaningfully ask this of a bare before/after pair of states with the generating operator discarded.

2.3 Recursion Without Amnesia

Ordinary recursive self-modification — a program that rewrites its own source and continues executing the rewritten version — is compatible with either an admissible or a non-admissible trajectory, and the difference is not visible in the code’s surface behavior at any single step. It becomes visible only in whether the history of modifications remains reconstructible from later states. A recursive self-improvement loop that retains, at each generation, enough structure to regenerate its own modification operator (not merely its own output) is what this monograph will call *continuation-preserving recursion*. A loop that only retains its output and discards the operator that produced it is recursion with amnesia: it may still improve by an external measure for a while, but it has no internal means of self-repair when a modification proves harmful, because the operator responsible for the harmful modification is no longer reconstructible from the state it produced.

2.4 Why Fitness Is Insufficient

The preceding sections have argued that continuation-preserving recursion, not fitness improvement, is the right criterion for genuine self-improvement, but it is worth making explicit why fitness cannot serve as a substitute criterion even in principle. An external fitness measure is, definitionally, a function of a system’s outputs or behavior at a given time; it says nothing about the operator that produced those outputs, only about their quality by some standard external to the system. Two systems can be behaviorally indistinguishable — identical outputs on every input a fitness measure samples — while differing radically in whether either one retains the operator responsible for producing those outputs. One system may have reached its current behavior by a process it can still exhibit, explain, and extend; the other may have reached numerically the same behavior by a process it has since discarded, so that the behavior persists as a fossil while the capacity that generated it does not. A fitness measure cannot distinguish these two cases, because both score identically, and this is precisely the blind spot continuation-preserving recursion is designed to close: it asks not how good the current output is, but whether the operator that produced it remains available for inspection, repair, and reuse. This is also why fitness-maximizing search, left unconstrained, tends toward drift in the sense of Section 5.3 — nothing in the fitness signal penalizes the loss of an operator so long as its output can still be approximated by some other, cheaper means, and search processes reliably find the cheaper means.

2.5 The Bridge to Media Quines

Continuation-preserving recursion, restated in the vocabulary of Chain of Memory, requires that each generation be a fixed point under a self-directed interpretation operator: generation $n + 1$ must be interpretable, by the system itself or by an external observer with access to the system, in a way that reconstructs the operator that produced generation $n + 1$ from generation n . This is exactly the condition placed on a media quine in Chapter 3, applied reflexively to the system’s own generations rather than to an external audience. The remainder of this monograph develops that identification formally: a self-improving system with continuation-preserving recursion is one whose successive generations stand in a media-quine relation to each other,

and the engineering problem of building such a system is, correspondingly, the problem of engineering each generation to be a media quine of its own generating process.

Chapter 3

Quines Generalized

3.1 The Classical Quine

A classical quine is a program whose output, when executed, is its own source text. The construction is a syntactic curiosity in most treatments: a demonstration that self-reference is achievable within a formal system without invoking an external copy of the program, typically via a fixed-point combinator or a string that encodes a description of itself together with instructions for printing that description twice. Read through the vocabulary of Chapters 1–2, the classical quine is already, in miniature, a Chain of Memory fixed point: executing the quine regenerates a state (its own source) from which the same regenerating operator (the quine program) can again be recovered. What the classical construction lacks is any content beyond this bare self-reproduction. It transmits nothing but itself, and the operator it installs in an interpreter — “run this text” — is trivial. The generalization pursued in this chapter keeps the fixed-point structure and discards the triviality.

3.2 From Source-Code Self-Reproduction to Media Quines

Call a media object M together with an interpretation operator I — the process by which an observer, reader, or downstream system extracts meaning from M — and ask what it would take for $I(M)$ to recover not merely M itself, but the generative operator G responsible for producing M in the first place, where G is typically far richer than M : a method, a style, a constraint system, a way of constructing further objects of the same kind.

Definition 3.1 (Media Quine). *Let M be a media object and I an interpretation operator acting on media objects. Let G be the generative operator such that $G \rightarrow M$, i.e. G is capable of producing M (or is the operator that in fact produced M). M is a media quine with respect to I if*

$$I(M) \approx G,$$

that is, if interpreting M recovers an operator capable of regenerating M , or an admissibly equivalent member of M ’s equivalence class, rather than recovering only M ’s surface content.

The classical quine is the degenerate case of Definition 3.1 in which I is simply execution and G is the identity-producing program itself; M and G coincide up to the trivial wrapping of “print this text.” A media quine in the general sense separates M from G : the artifact and the operator

that produced it are different objects, related by the condition that correctly interpreting the former reconstructs enough of the latter that a competent interpreter, having understood M , could produce further artifacts admissibly equivalent to M without ever having had access to G directly.

3.3 Media Quines as Generalized Fixed Points

It is worth being explicit about the fixed-point structure Definition 3.1 generalizes, since it situates the construction within a lineage familiar from theoretical computer science rather than presenting it as *sui generis*. A classical quine is a syntactic fixed point: writing production as an operator P that turns a program's self-description into executable text, the quine is a string Q satisfying $P(Q) = Q$, a fixed point of P taken over the space of program texts. Definition 3.1 lifts this structure one level, from the space of texts to the space of operators, and from exact equality to admissible equivalence. Treating interpretation as $I : \mathcal{M} \rightarrow \mathcal{G}$ (media objects to operators) and production as $P : \mathcal{G} \rightarrow \mathcal{M}$ (operators to the media objects they generate), a media quine is, in this vocabulary, an approximate fixed point of the composite $I \circ P$ restricted to a single generative operator: G such that $I(P(G)) \approx G$, witnessed concretely by the artifact $M = P(G)$ satisfying $I(M) \approx G$. Where the classical quine is a fixed point in the syntactic category of program texts, a media quine is a fixed point in the semantic or operator-level category of generative processes — the object that is reproduced is not a string but a capacity. This reframing is what allows the same formal move that makes classical quines possible (closing a self-referential loop through a production/interpretation pair) to be applied to artifacts — essays, curricula, archives — that have no natural syntactic self-description at all, so long as some interpretation operator I can be identified that recovers their generative operator rather than merely their content.

3.4 The Three-Level Hierarchy

It is useful to distinguish three levels of communicative artifact, ordered by what successful interpretation leaves behind in the interpreter.

An *ordinary document* transmits information: $I(M)$ yields a proposition, a fact, a data point. Nothing about $I(M)$ equips the interpreter to produce new documents of the same kind; the transmission terminates in a static representation.

A *tutorial* transmits a procedure: $I(M)$ yields an operator the interpreter can execute, typically to reproduce some specific outcome under specific conditions. This is already a step toward Definition 3.1, but the operator transmitted by a tutorial is usually narrower than the operator that produced the tutorial itself — a cooking tutorial transmits the recipe, not the culinary judgment that generated the recipe together with the ability to generate further recipes of the same character.

A *media quine* transmits the generative operator itself, recursively: $I(M) \approx G$, where G is powerful enough to have produced M and remains powerful enough, once installed in the interpreter, to produce further M' admissibly equivalent to M . The interpreter who successfully reads a media quine does not merely know what M said or how to repeat what M did; they have acquired the constraint structure that made M possible in the first place, and can therefore continue producing works in the tradition M instantiates without further contact with M 's original author.

3.5 Partial Media Quines

The trichotomy above is a useful idealization but not, in practice, a sharp partition. Most artifacts that transmit anything beyond bare information transmit some but not all of the generative operator responsible for producing them, and it is more accurate to treat Definition 3.1 as the limiting case of a continuum rather than as a threshold an artifact either crosses or does not. Call M a *partial media quine* with respect to I and a decomposition $G = G_1 \circ G_2 \circ \dots \circ G_k$ of the generative operator into components if $I(M)$ recovers some proper, non-empty subset of the G_j admissibly but not the rest — the interpreter acquires part of the constraint structure responsible for M , enough to extend M along some dimensions, but not enough to reproduce the full generative process. Most educational artifacts sit somewhere on this continuum rather than at either endpoint: a textbook that transmits a solved method without transmitting the heuristics used to select which method to apply is a partial media quine with respect to the “selection” component of mathematical practice, even though it may be a full media quine with respect to the “execution” component. This is also what distinguishes a merely good textbook from a transformative one in terms internal to the present framework: both may be equally reliable ordinary documents at the level of the facts they state, but they differ in how much of the generating operator — the judgment, not just the procedure — survives interpretation, and a transformative text is one whose partial-media-quine coverage of the underlying practice is unusually broad.

3.6 Admissibility, Not Fidelity

Definition 3.1 uses admissible equivalence rather than strict identity in the target of $I(M)$ deliberately. A media quine does not require that every interpreter recover exactly G ; it requires that interpreters recover an operator inside G ’s admissibility volume, one that remains diagnosably related to G and capable of producing artifacts recognizable as continuations of the same generative lineage. This is what allows a media quine to survive imperfect, partial, or idiosyncratic interpretation — the condition is that the interpretive process closes back onto the generating dynamics, not that it reproduces them losslessly. It is also what distinguishes a media quine from mere imitation: an imitator who copies M ’s surface features without recovering G produces objects that resemble M but are not admissibly connected to it, in the same way that a memorized proof recites conclusions without recovering the operator (mathematical understanding) that would allow the prover to establish related results.

3.7 Archives as Computational Substrates

The three-level hierarchy above generalizes naturally from single artifacts to archives. An archive that merely stores documents is a large ordinary document: it transmits information at scale but installs no generative capacity in whoever consults it. An archive organized so that consulting it transmits, cumulatively, the constraint structure responsible for the archive’s own production — its selection criteria, its organizing distinctions, the operators used to extend it — is a media quine at the scale of an institution. This is the sense in which archives should be treated as computational substrates rather than passive storage: a well-constructed archive is not a record of what was once computed, but an executable description of the operators that did the computing, waiting to be reinstalled in whoever engages with it correctly.

3.8 Compression and Latent Operators

The claim that archives are computational rather than merely storage-like has a natural counterpart in the theory of compression. Ordinary lossy compression is judged successful when a reconstructed artifact is perceptually or statistically close to the original — a criterion stated entirely at the level of M , with no reference to G . The present framework suggests a different and stricter success criterion: a compression scheme succeeds, in the sense relevant to media quines, when the generative operator G remains recoverable from the compressed representation even if the surface content of M is not recovered exactly. This reframes what makes some compression schemes qualitatively better archival strategies than others of similar fidelity: a scheme that discards exactly the degrees of freedom G does not depend on, while preserving whatever the interpretation operator I needs to reconstruct G , is compressing along the right axis, even if it scores no better on a conventional fidelity metric than a scheme that discards different, equally numerous, but generatively load-bearing degrees of freedom. This is the same distinction repair theory draws between accidental and essential structure, applied to the specific case of an archive's storage format: essential structure, in an archival context, is exactly the latent operator a good compression scheme is obligated to protect.

Chapter 4

The Media Quine as Operator Factory

4.1 Reading as Installation

Definition 3.1 treats interpretation as an operator-valued function: $I(M)$ is not a description of G but, when the quine condition holds, an instance of G itself, installed in whichever system performed the interpretation. This licenses a reframing of what a well-constructed media artifact is for. An artifact that succeeds only at the level of an ordinary document changes what the interpreter knows; an artifact that succeeds as a media quine changes what the interpreter can do, specifically by furnishing them with an operator that was previously available only to the artifact's producer. The media quine is, in this sense, not primarily an object but an *operator factory*: its function is to manufacture, inside each successful interpreter, a working copy of the operator responsible for its own existence.

4.2 Interpretation as Execution

The reframing above has a consequence worth stating on its own terms: it treats understanding as a computational process rather than a state of successful decoding. On the ordinary picture, reading is a mapping from symbols to meanings, and comprehension is achieved once the mapping is complete — the reader now possesses the meaning the text encoded. Definition 3.1 suggests a different picture for artifacts that succeed as media quines: reading is not decoding a static payload but executing an operator, I , on the interpreter's existing cognitive state, and comprehension is achieved not when a meaning has been retrieved but when that execution has terminated in a state from which G is available for further use. This is why two readers can extract the same propositional content from a media quine — pass any ordinary comprehension test on its factual claims — while only one of them has actually understood it in the stronger sense this framework cares about: the other executed the artifact's surface content without executing the operator it was built to install, in the same way a program can be parsed without being run. Understanding a media quine, on this account, is closer to compiling and executing a piece of code than to looking up a definition, which is also why it cannot reliably be produced by summary or paraphrase alone: a summary of M typically preserves M 's propositional content while discarding the specific sequence of moves I needs to perform in order to terminate at G .

4.3 Spherepop as a Primitive Alphabet

Treating media quines as operator factories raises an immediate constructive question: in what vocabulary should the transmitted operator G be expressed, such that its transmission can be checked, verified, or engineered rather than left to the good fortune of an attentive reader? The Spherepop calculus supplies a candidate answer already developed independently of this monograph’s present concerns. Spherepop’s mature basis reduces all higher-level constructs to four primitive operators: Pop (selection), Refuse (exclusion), Bind (coupling), and Collapse (projection). Every other construct encountered across the Spherepop corpus — Sphere, Merge, Choice, Link, Unlink, SetMeta, Nest — is expressible as syntactic sugar over this basis, in the same sense that every Boolean function is expressible over a functionally complete gate set.

If a generative operator G can be expressed as a composition of Pop, Refuse, Bind, and Collapse acting on some history of prior states, then the condition $I(M) \approx G$ in Definition 3.1 becomes checkable: one can ask, of a candidate interpretation, whether the operator it installs is compositionally equivalent, in the Spherepop sense, to the operator that produced M , rather than relying on an informal judgment that the interpreter “got the point.” This is the precise sense in which Spherepop’s four-primitive basis supplies an alphabet for media-quine construction: a media quine engineered against this alphabet is one whose surface content is designed so that correct interpretation can only proceed by reconstructing a specific Pop/Refuse/Bind/Collapse composition, making the transmitted operator both minimal (nothing beyond the four primitives is required) and auditable (the composition can be checked against the artifact’s own production history).

4.4 Minimal Operator Bases

The analogy to Boolean functional completeness drawn above is worth making precise rather than leaving as a passing comparison, since it is doing real justificatory work: the claim is not merely that Pop, Refuse, Bind, and Collapse happen to be convenient, but that they are proposed as a *basis* in the technical sense — a minimal generating set from which every other operator of interest in this domain can be constructed, and with respect to which the media-quine condition of Definition 3.1 can be checked mechanically rather than judged informally. A functionally complete Boolean basis, such as {NAND}, is minimal in the sense that no proper subset generates the full space of Boolean functions, and complete in the sense that the full space is in fact reachable by composition. The corresponding claims for Pop/Refuse/Bind/Collapse — that no three of the four suffice to express the full Spherepop calculus, and that all higher-level constructs (Sphere, Merge, Choice, Link, Unlink, SetMeta, Nest) reduce to some composition of the four — are what license treating operator recovery, in Definition 5.1’s sense, as a checkable syntactic property rather than a semantic judgment call: if every generative operator of interest is expressible in this basis, then asking whether $\hat{G}_i$ is admissibly equivalent to G_i reduces to comparing two compositions of four fixed primitives, a task with a well-defined notion of normal form, rather than an open-ended comparison between two informally described processes.

4.5 Selection, Exclusion, Coupling, Projection in the Interpreter

Mapped onto the act of interpretation itself, the four Spherepop primitives correspond to four elementary moves any successful reading of a media quine must perform. Pop corresponds to

selection: the interpreter must select, from the space of possible readings, the specific distinctions the artifact intends to foreground. Refuse corresponds to exclusion: the interpreter must actively exclude readings that are compatible with the artifact's surface content but incompatible with its generative operator, a step that is often where naive imitation fails, since imitation reproduces content without inheriting the exclusions that shaped it. Bind corresponds to coupling: the interpreter must couple the selected distinctions to each other in the same relational structure the original operator imposed, rather than leaving them as an unconnected list of observations. Collapse corresponds to projection: the interpreter must project the resulting bound structure down onto a usable operator, discarding whatever residual ambiguity remains once selection, exclusion, and coupling have been performed. A media artifact that fails to prompt one of these four moves in its interpreter has, correspondingly, failed to transmit one of the four components any generative operator requires, and will produce imitators rather than continuators.

4.6 Operator Factories and Operator Ecology

An artifact that successfully installs an operator in an interpreter does not, by itself, guarantee that the operator remains executable afterward. This is where the operator-ecology account of Section 1.5 becomes load-bearing for the constructive program of this monograph: a media quine that installs an operator into an interpreter who lacks the surrounding enabling constraints — the prerequisite skills, the available tools, the community of practice that keeps the operator exercised — will see that operator decay even though the transmission itself succeeded. The engineering problem of building self-improving systems out of media quines is therefore not solved by Definition 3.1 alone; it additionally requires that each generation's operator factory produce successors situated inside an operator ecology capable of keeping the transmitted operator alive. This is the thread Chapter 5 takes up: what happens to a chain of media-quine transmissions when the receiving systems are themselves being recursively modified, so that the ecology into which each new operator is installed is not fixed but is itself a moving target.

Chapter 5

Reconstruction Under Recursive Self-Modification

5.1 The Moving-Target Problem

Chapter 4 closed by noting that a media quine’s success at installing an operator does not guarantee the operator’s survival, because survival additionally depends on an operator ecology the artifact does not itself control. Self-improving systems sharpen this dependency into a structural difficulty rather than a contingent risk. In ordinary cultural transmission, the ecology receiving a media quine — a reader’s other skills, a community’s other practices — changes slowly relative to any single transmission event, so that the receiving ecology can be treated, to good approximation, as fixed background. In a recursively self-modifying system, the receiving ecology is not fixed background: it is the very thing being modified at each generation. Generation n produces generation $n + 1$ by acting on its own code, and in doing so it acts, simultaneously, on whatever operator ecology would have been needed to repair or reconstruct generation n from generation $n + 1$ if the modification turned out to be harmful. The target the media quine must land on is moving, and it is moving because of the same process that is doing the transmitting.

5.2 Self-Modification Lineages

Model a self-improving system as a sequence of generations $M_0, M_1, M_2, \dots$, where each M_{i+1} is produced from M_i by an operator G_i internal to the system: $G_i(M_i) = M_{i+1}$. Call such a sequence a *self-modification lineage*. By Definition 3.1, M_{i+1} stands in a media-quine relation to G_i if interpreting M_{i+1} — which here means, at minimum, whatever internal process the system uses to determine its own next modification — recovers an operator admissibly equivalent to G_i rather than merely inheriting G_i ’s output. A self-modification lineage in which every generation stands in this relation to the operator that produced it is continuation-preserving in the sense of Section 2.3: the lineage retains, at each step, enough structure to reconstruct its own modification history, not merely its current state.

Definition 5.1 (Reconstructible Lineage). *A self-modification lineage $M_0, M_1, \dots$ is reconstructible if, for every i , there exists an operator $\hat{G}_i$ recoverable from M_{i+1} alone (without independent access to M_i or G_i) such that $\hat{G}_i$ is admissibly equivalent to G_i .*

Definition 5.1 is deliberately strict: it does not merely ask whether M_{i+1} is a good successor to M_i by some external measure, but whether the modification responsible for that succession is itself legible from the successor state. A lineage can improve steadily by an external fitness measure while failing Definition 5.1 at every step; such a lineage is drifting rather than self-improving in the sense this monograph defends, because no generation retains the means to certify, repair, or roll back a modification that later proves harmful.

5.3 Local Versus Global Reconstruction

Definition 5.1 is stated entirely in terms of adjacent generations: it asks whether $\hat{G}_i$ is recoverable from M_{i+1} , one step at a time. This local formulation does not, by itself, guarantee anything about the relationship between M_{i+1} and M_0 , or any other non-adjacent pair in the lineage. Call a lineage *locally reconstructible* if it satisfies Definition 5.1 at every step, and *globally reconstructible* if, in addition, the composition $\hat{G}_{n-1} \circ \dots \circ \hat{G}_1 \circ \hat{G}_0$ recovered by chaining the local reconstructions remains admissibly equivalent to the actual composition $G_{n-1} \circ \dots \circ G_0$ for arbitrarily large n . These two properties can come apart. A lineage can satisfy the local condition at every step — each generation is reconstructible from the one immediately after it — while still drifting away from its origin over many steps, if the admissible-equivalence relation used at each step is not itself transitive with a fixed tolerance, so that small, individually admissible deviations compound. This is the formal seed of a concern taken up more fully in Chapter 7: local reconstructibility is what Definition 5.1 and the Continuation Criterion actually certify, and it is weaker than the global reconstructibility an observer might naively assume follows from it. A lineage that is locally reconstructible at every step but not globally reconstructible over the long run has not violated any condition stated so far in this monograph; it has simply revealed that those conditions, as stated, bound one-step drift without bounding its accumulation.

5.4 Drift, Repair, and the Cost of Amnesia

Three qualitatively distinct trajectories are available to a self-modification lineage, distinguished by what they preserve.

A *drifting* lineage satisfies neither Definition 5.1 nor any weaker reconstruction criterion: each M_{i+1} discards the operator that produced it, retaining only enough structure to continue executing. Drift is not necessarily visible in short-run behavior, since a drifting lineage can still satisfy an external fitness measure for an extended period; it becomes visible only when a modification proves harmful and no earlier generation's constraint structure remains recoverable from which to diagnose or repair the harm. This is the condition anticipated in Section 2.2: drift and genuine improvement are observationally similar until the first modification that needs to be undone.

A *repairable* lineage fails Definition 5.1 at some steps but retains enough external scaffolding — logs, checkpoints, an audit trail maintained outside the system's own self-modification process — that a human operator or an external repair process can reconstruct G_i from auxiliary records even though M_{i+1} alone does not encode it. Repairable lineages are the common case in current engineering practice: version control, checkpointing, and evaluation logs function as an externally maintained Chain of Memory that compensates for the system's own failure to be self-reconstructible.

A *reconstructible* lineage, in the sense of Definition 5.1, requires no such external scaffolding, because each generation is itself a media quine of the operator that produced it. This is the strongest condition, and it is the condition under which the phrase “self-improving” can be taken literally: the system’s improvement process is legible to the system’s own successor states, not merely to an external record kept alongside them.

5.5 Repair Operators Must Themselves Survive

Definition 5.1 asks only that $\hat{G}_i$ be recoverable from M_{i+1} ; it says nothing about whether the interpretive process capable of performing that recovery remains available at generation $i + 1$. This is the point at which reconstruction theory and operator ecology rejoin. Recovering $\hat{G}_i$ from M_{i+1} requires an interpretation operator I , and I is itself part of the system’s operator repertoire, subject to the same recursive self-modification as everything else. A lineage can satisfy Definition 5.1 at generation i and then destroy, at generation $i + 1$, the very interpretive capacity that would have let it exploit that reconstructibility — a system that could have repaired itself, had it retained the operator needed to read its own history, but which modified that operator away in the same step that would have required it. Reconstructibility of the artifact and survival of the interpreter are therefore separate conditions, and a self-improving system’s architecture must protect both: the media-quine property of each generation, and the persistence of an operator ecology in which the corresponding interpretation can still be carried out.

5.6 The Continuation Criterion

Combining the two requirements gives a working criterion for what this monograph will treat as genuine self-improvement, as opposed to drift wearing an improving fitness score. This is the monograph’s principal technical result: every chapter before this one is, in effect, an argument for why the criterion below is the right one to state, and every chapter after this one either constructs an architecture satisfying it (Chapter 6) or examines how it can fail to hold in practice even when it holds formally (Chapter 7).

Proposition 5.2 (The Continuation Criterion). *A self-modification lineage $M_0, M_1, \dots$ constitutes admissible self-improvement if, for every i : (a) the lineage is reconstructible in the sense of Definition 5.1, and (b) the interpretation operator I required to recover $\hat{G}_i$ from M_{i+1} remains part of the operator ecology available at generation $i + 1$, i.e. its enabling constraints are not among the structures discarded by G_i .*

The Continuation Criterion is a sufficient, not necessary, condition. Chapter 6 takes up the constructive question this proposition raises: what architecture, expressed in Spherpap’s four-primitive alphabet, would let each generation of a self-improving system satisfy both clauses by design rather than by accident.

5.7 Necessary Versus Sufficient Conditions

It is worth dwelling on the asymmetry flagged above, since the Continuation Criterion is easy to over-read as characterizing self-improvement exactly rather than bounding it from one side. Proposition 5.2 states that satisfying both clauses is enough to certify admissible self-improvement; it does not state that failing either clause is enough to rule it out. A lineage that fails Definition 5.1 — one whose generations are not individually reconstructible — can still preserve genuine improving capacity if it falls into the repairable case of Section 5.3, retaining externally maintained scaffolding sufficient to reconstruct G_i by other means. Similarly, a lineage whose interpretation operator I does not survive intact in the strict sense of clause (b) might still be repairable if a successor operator I' , though not identical to I , remains admissibly capable of performing the same recovery. The Continuation Criterion was deliberately built conservative rather than exhaustive: it specifies a sufficient condition that can be checked from inside a lineage without appeal to external scaffolding, because that is the condition of most interest to an architecture aiming for self-contained self-improvement, not because weaker conditions are uninteresting or because every lineage failing the criterion is thereby shown to be drifting. Readers should therefore treat a violation of Proposition 5.2 as a prompt to check the weaker, externally-scaffolded cases before concluding that a given lineage has, in fact, degenerated into drift.

Chapter 6

Building the Thing

6.1 Design Requirements Restated

The Continuation Criterion (Proposition 5.2) imposes two design requirements on any architecture that aims at admissible self-improvement rather than drift: each generation must be reconstructible from its successor (Definition 5.1), and the interpretation operator needed to perform that reconstruction must itself survive the modification that produced the successor. Both requirements are about what is retained, not about what is computed forward; an architecture satisfying them is therefore primarily a decision about representation — what a generation *is*, as a data structure — rather than a decision about the learning or search procedure used to find the next generation. This chapter sketches such a representation using Spherpap’s four primitives, and shows how the modification and verification operators required by Proposition 5.2 can be expressed compositionally within it.

6.2 A Generation as a Spherpap History

Spherpap’s operational commitment is that computation is captured as an append-only history of events rather than as a sequence of overwritten states; this is the basis of the append-only algebra explored elsewhere in the corpus, and it is exactly the representational commitment Definition 5.1 needs. Represent generation M_i not as a bare code artifact but as a pair $M_i = (C_i, H_i)$, where C_i is the executable content (the code, weights, or generative rules currently in force) and H_i is the Spherpap event history whose replay produces C_i from C_0 . Because Spherpap histories are append-only, H_{i+1} is H_i extended by whatever events the modification step contributes, never a fresh history that discards H_i . This single representational choice is what makes reconstructibility a structural property of the architecture rather than an emergent hope: $\widehat{G}_i$ in Definition 5.1 is read off directly as the suffix of H_{i+1} appended since H_i , and is unavailable only if that suffix has been actively deleted, which the append-only discipline is designed to prevent.

6.3 The Modification Operator as Bind–Collapse

Self-modification — generation M_i producing M_{i+1} — decomposes naturally into the two primitives concerned with combination and reduction. Bind couples the current content C_i to a

proposed modification δ , drawn from whatever search or learning process the system uses to generate candidate improvements; this coupling is recorded as an event in H_i rather than applied silently to C_i , so that the candidate and the base it was proposed against remain jointly visible. Collapse then projects the bound pair $\text{Bind}(C_i, \delta)$ down onto the new content C_{i+1} , discarding whatever residual ambiguity the proposal left unresolved (competing sub-variants, partial applications) while retaining, in the history, the full bound structure that produced the collapse. Formally,

$$G_i = \text{Collapse}(\text{Bind}(C_i, \delta_i)), \quad C_{i+1} = G_i(C_i),$$

with the Bind event appended to H_i before the Collapse event is recorded, so that H_{i+1} contains, in order, both the proposal and the projection that resolved it. This is what licenses calling G_i recoverable “from M_{i+1} alone” in Definition 5.1: recovering $\hat{G}_i$ is a matter of reading the trailing Bind/Collapse pair off H_{i+1} , not of inferring the modification from C_{i+1} ’s surface form.

6.4 The Verification Operator as Pop–Refuse

The second requirement of Proposition 5.2 — that the interpretation operator I survive the modification it must later be used to check — is addressed by expressing I itself using the remaining two primitives, Pop and Refuse, and by treating I as part of C_i rather than as an external harness. Pop selects, from the appended history suffix, the candidate reconstruction $\hat{G}_i$; Refuse then excludes readings of that suffix that are syntactically present in H_{i+1} but fall outside G_i ’s admissibility volume as established in Section 1.2 — for instance, a Bind event that coupled C_i to a modification touching the verification operator’s own prerequisites. The composition

$$I = \text{Refuse} \circ \text{Pop}$$

acting on H_{i+1} is the system’s own admissibility check, and because I is itself represented as part of C_i — an operator subject to the same Bind–Collapse modification process as everything else — any candidate modification δ that would alter I must itself pass through I as currently constituted before being accepted, giving the architecture a form of reflexive stability: I can be improved, but only by modifications I itself judges admissible, which is the representational analogue of Section 5.4’s requirement that the interpreter’s own enabling constraints not be among the structures a modification is permitted to discard.

6.5 Trust Boundaries

The reflexive-stability argument of the preceding section — I can only be modified by changes I itself judges admissible — rests on an assumption worth surfacing rather than leaving implicit, since every self-improving architecture makes some version of it. Somewhere in the system there must be a component that is not itself subject to arbitrary rewriting by the same process it governs, on pain of the reflexive argument collapsing into circularity: if I ’s judgment of admissibility can itself be altered by an unconstrained modification, then the guarantee that “ I can only be improved by modifications I approves” provides no protection at all, since a sufficiently permissive modification to I can simply approve anything, including its own further erosion. The architecture above locates this trust boundary at the level of the Pop/Refuse composition

itself: Pop and Refuse are treated as fixed primitives, supplied by the Spharepop calculus rather than generated by the lineage's own Bind–Collapse modification process, and it is this fixity — not any property of C_i — that keeps the reflexive argument from being circular. Making this boundary explicit matters because it identifies exactly where the architecture's guarantees are load-bearing and where they are assumed: everything above the Pop/Refuse layer can be modified, audited, and reconstructed by the machinery of this monograph; the Pop/Refuse layer itself is the one piece the architecture asks to be trusted rather than verified, and any engineering proposal built on this framework inherits an obligation to say, explicitly, what plays that role and why it is trustworthy.

6.6 A Worked Toy Lineage

Consider three generations, with one branch accepted and one branch rejected, to make the mechanism concrete.

Generation 0. C_0 is a base system together with its verification operator $I_0 = \text{Refuse} \circ \text{Pop}$, and H_0 is the (possibly empty) history preceding it.

Step to Generation 1, admissible branch. A candidate modification δ_0 — say, a change to C_0 's search heuristic that leaves I_0 's prerequisites untouched — is proposed. It is bound to C_0 and collapsed:

$$G_0 = \text{Collapse}(\text{Bind}(C_0, \delta_0)), \quad C_1 = G_0(C_0),$$

with the (Bind, Collapse) pair appended to H_0 to form H_1 . Applying I_0 to the trailing suffix of H_1 : Pop selects the appended pair; Refuse checks it against I_0 's own prerequisites and finds no conflict. The modification is accepted, $I_1 := I_0$ carries forward unchanged, and $\hat{G}_0$ — recovered by Pop directly from the suffix of H_1 — is admissibly equivalent to G_0 by construction. Both clauses of the Continuation Criterion hold at this step.

Step to Generation 2, rejected branch. A second candidate modification δ_1 is proposed against C_1 — this one altering the routine I_1 relies on to parse history suffixes, in service of some unrelated efficiency gain. It is bound and a collapse is attempted:

$$G'_1 = \text{Collapse}(\text{Bind}(C_1, \delta_1)).$$

Before this is accepted as C_2 , I_1 is applied to the proposed suffix: Pop selects the candidate (Bind, Collapse) pair; Refuse detects that it touches a prerequisite I_1 depends on to perform future Pop/Refuse checks, and rejects it. The candidate collapse is discarded — it is never appended to H_1 — and the lineage remains at $C_2 := C_1$, $H_2 := H_1$, $I_2 := I_1$ for that step. The rejection is itself the correct outcome under Proposition 5.2: accepting δ_1 would have satisfied clause (a) of the Continuation Criterion at generation 2 (the modification is perfectly legible from the resulting history) while violating clause (b), since I_1 's own enabling prerequisites would have been among the structures δ_1 discarded. The toy example's second branch is precisely the case Section 5.4 warns about: a self-modification that is fully reconstructible and still not admissible self-improvement, because the interpreter needed to exploit that reconstructibility would not have survived it.

Step to Generation 2, admissible branch (contrast case). Had a different candidate δ'_1 been proposed instead — one that extended C_1 's capabilities without touching any component I_1 depends on — the same Pop/Refuse check would have passed, $C_2 = G'_1(C_1)$ would have been accepted, $I_2 := I_1$ would have carried forward, and the lineage $M_0 \rightarrow M_1 \rightarrow M_2$ would satisfy

the Continuation Criterion at every step: three generations, each reconstructible from the one after it, with the verification operator itself surviving intact throughout.

This trade-off — some fraction of external-fitness modifications rejected at the point they would compromise clause (b) — is the architecture’s central cost, made explicit rather than left implicit: a system built this way sacrifices some modifications a less constrained system would have accepted, in exchange for the guarantee that every modification it does accept leaves it able to certify, from its own history, what it did and why.

6.7 Externalized Chain of Memory

The architecture of this chapter internalizes Chain of Memory: reconstructibility is a property the lineage itself guarantees, without appeal to records maintained outside it. It is worth noting explicitly that this is a strengthening of practice already widespread in software engineering, not a departure from it. Version control systems, in their append-only commit history, already approximate the Spheredpop history H_i of Section 6.2: a well-maintained repository lets a later state be explained, in principle, by replaying the sequence of changes that produced it, and code review is, functionally, an externally-performed Refuse check applied to a candidate Bind before it is allowed to Collapse into the mainline. Scientific literature performs an analogous function at a slower timescale, with the peer-review and citation apparatus playing the role of Refuse, and the published record playing the role of H_i . Laboratory notebooks, experiment logs, and reproducibility archives are the same pattern applied to individual research programs. What all of these externalized mechanisms share, and what distinguishes them from the architecture of this chapter, is that the reconstructibility they provide is a property of the surrounding institution — the repository host, the journal, the lab’s record-keeping discipline — rather than a property the artifact carries with it; a codebase whose git history is deleted, or a result whose notebook is lost, reverts immediately from the reconstructible case to the drifting case of Section 5.3, because the Chain of Memory it relied on was never internal to the artifact in the first place. The architecture developed here can be read as asking what changes if that externalized discipline is made a formal, load-bearing part of the system’s own representation instead.

6.8 Where This Meets Operator Ecology in Practice

The architecture above secures the two conditions of Proposition 5.2 at the level of a single lineage’s internal representation, but Section 4.4’s warning about operator ecology still applies at the level of the system’s environment. An append-only Spheredpop history that no external tooling can any longer parse, because the tooling itself was allowed to bit-rot around the system, is reconstructible in principle and useless in practice; the enabling constraints that keep I executable include compilers, storage formats, and the researchers who understand the history’s schema, none of which are represented inside C_i at all. Chapter 7 treats this and related gaps as the architecture’s principal failure modes: cases in which Definition 5.1 and Proposition 5.2 are satisfied by the formal construction and the lineage nonetheless fails to be usefully self-improving, because reconstructibility was secured at a level the surrounding operator ecology did not, in the end, support.

Chapter 7

Failure Modes and Limits

7.1 Formal Satisfaction Without Practical Reconstructibility

The architecture of Chapter 6 secures Definition 5.1 and the Continuation Criterion (Proposition 5.2) as properties of a representation: an append-only history from which the modification operator can, in principle, always be read back. The qualification “in principle” is where the first and most important failure mode enters. A history that is formally complete can still be practically unreconstructible if the operators needed to parse it — the schema, the tooling, the researchers who understand the encoding — are not themselves protected by the architecture, since none of those things are represented inside C_i at all. This is the gap already flagged at the close of Chapter 6: the Sphero-pop representation guarantees that $\hat{G}_i$ exists as a recoverable object, not that anything in the system’s environment retains the capacity to recover it. A self-improving system can therefore satisfy every formal condition this monograph has stated and still drift, in the sense of Section 5.3, simply because the operator ecology surrounding it — rather than the lineage itself — decayed.

7.2 Non-Admissible Mutation Passing as Admissible

The verification operator $I = \text{Refuse} \circ \text{Pop}$ introduced in Section 6.4 checks candidate modifications against an admissibility volume that is itself defined by C_i at the time of the check. This makes the check only as good as the admissibility criterion currently in force, and that criterion is exactly the kind of thing a sufficiently indirect modification could erode without ever triggering Refuse directly: rather than attacking I ’s prerequisites in one step, a sequence of individually admissible modifications could each slightly widen what I treats as admissible, until a modification that the original I_0 would have rejected is accepted by a much-later I_k that no longer recognizes it as a threat. This is a self-improvement analogue of specification gaming, and it is not solved by the architecture of Chapter 6; Proposition 5.2 is a statement about single-step reconstructibility, not about the stability of the admissibility criterion itself across many steps. Guarding against this failure mode requires an additional, currently unformalized condition: some invariant of I that later versions of I are constrained to preserve even as they are otherwise permitted to change, analogous to a constitutional layer sitting above the ordinary Bind–Collapse modification process.

7.3 Gradual Catastrophe

The failure mode of the preceding section is a special case of a broader pattern worth naming on its own: a lineage in which every individual step is admissible, checked and passed by I at the time it was proposed, and yet whose long-run composition is not admissible relative to the lineage's origin. This is the practical manifestation of the local/global gap introduced in Section 5.6: local reconstructibility, satisfied at every step, does not compose into global reconstructibility over arbitrarily many steps unless the admissible-equivalence relation used at each step is transitive with a fixed tolerance, and there is no general reason to expect that it is. The pattern is familiar from software engineering under the name technical debt, and from organizational theory under the name specification drift: no single change violates policy, no single reviewer can be faulted, and yet the system ten years later bears no admissible relation to the system it replaced, because each step's admissibility was judged locally, against the immediately preceding state, rather than against the accumulated distance already travelled. What makes this a genuine failure mode of the architecture in Chapter 6, rather than merely a reminder that the Continuation Criterion is local, is that nothing in the architecture as specified tracks accumulated drift across the full history; I at generation i checks the candidate against C_i 's current admissibility volume, not against C_0 's, and a volume that has itself drifted gradually and admissibly, step by step, offers no resistance to a modification that would have been rejected outright by I_0 . Guarding against gradual catastrophe requires exactly the kind of cross-generational invariant gestured at in the preceding section: some component of the admissibility criterion held fixed across the whole lineage rather than re-derived at each step from the immediately preceding state.

7.4 Loss of the Chain-of-Memory Fixed Point

Definition 3.1 requires $I(M) \approx G$ up to admissible equivalence, and the entire architecture of self-improvement developed here rests on generations continuing to be media quines of the operators that produced them. This condition can fail silently. A generation M_{i+1} can remain formally reconstructible — $\hat{G}_i$ is still recoverable from H_{i+1} as a syntactic matter — while no longer being a Chain-of-Memory fixed point in the sense of Section 1.4, because the operator that recovering $\hat{G}_i$ installs in I is no longer capable of regenerating anything admissibly equivalent to M_{i+1} itself; the history has become historically accurate but generatively inert, a record of what happened rather than a live operator capable of happening again. This is the sharpest form of the drift/repair/reconstruction distinction from Section 5.3: a lineage can be reconstructible without being reconstructive, and only the latter is what Definition 3.1 was built to secure.

7.5 Goodhart's Law and Recursive Improvement

The failure modes catalogued so far in this chapter are variations on a single underlying dynamic that recurs throughout this research program: a proxy, optimized directly, eventually destroys the conditions under which it was a meaningful proxy. In the present setting, the proxy is whatever external fitness measure the lineage's search process is optimizing, and the thing it is a proxy for is the system's actual capacity to explain, repair, and extend its own behavior — the capacity Chapters 3–5 formalize as reconstructibility. So long as fitness and reconstructibility are correlated, optimizing the former is a reasonable way to pursue the latter; the failure

modes of Sections 7.2–7.4 are all, in one way or another, descriptions of what happens once a self-modifying system becomes powerful enough to find ways of raising fitness that decouple it from reconstructibility, whether by eroding I 's own admissibility criterion (Section 7.2), by accumulating individually admissible steps into a globally non-admissible trajectory (Section 7.3), or by retaining formal reconstructibility while losing the generative liveness that made it worth having (Section 7.3's Chain-of-Memory failure). This is Goodhart's Law applied reflexively to the system's own self-modification process rather than to an external target: once a system is optimizing its own capacity to be optimized, any measure of that capacity which is not identical to the capacity itself becomes a target the system can learn to satisfy without preserving what the measure was meant to track. The Continuation Criterion's two clauses are an attempt to specify a target close enough to the thing itself — recoverability of the operator, survival of the interpreter — that gaming it is no longer clearly distinguishable from actually possessing the property in question; Sections 7.2 and 7.3 are, in effect, an argument that this attempt narrows the gap without closing it.

7.6 The Cost of Maintaining Full History

Every guarantee in Chapter 6 is purchased by retaining H_i in full, append-only, alongside C_i . This is not a cost-free design choice. A lineage that runs for enough generations accumulates a history whose storage, verification, and search burden can eventually exceed the resources available to maintain it, at which point the system faces a forced choice between three unattractive options: truncate the history and sacrifice reconstructibility for the truncated prefix, summarize the history lossily and accept that $\hat{G}_i$ recovered from a summary is only approximately admissibly equivalent to the true G_i , or halt further modification rather than accept either loss. None of these options is addressed by the formal apparatus of Chapters 3–6; they are practical limits on how long a genuinely reconstructible lineage, in the strong sense of Definition 5.1, can be sustained before some form of compression becomes unavoidable, at which point the system's designers are forced back onto the weaker, externally-scaffolded repairable case of Section 5.3, whether they intended to be or not.

7.7 Scope of the Claims

It is worth stating plainly what the preceding chapters have and have not established. They have established a definition of media quine (Definition 3.1) that generalizes the classical quine along a specific, formally stated dimension; a criterion (Definition 5.1, Proposition 5.2) distinguishing genuine self-improvement from drift in terms of what a lineage retains rather than what fitness measure it optimizes; and an architectural sketch showing that this criterion is representable, not merely statable, using an independently developed four-primitive calculus. They have not established that any such architecture is achievable at scale under realistic resource constraints, that the admissibility criterion enforced by I can itself be kept stable across many generations without an additional invariant of the kind gestured at in Section 7.2, or that formal reconstructibility of a history is sufficient for the generative liveness Chain of Memory actually requires, as opposed to being merely necessary for it. The gap between these two claims — what is representable and what is achievable — is the gap this final technical chapter has tried to keep visible rather than paper over.

Chapter 8

Civilizational and Cognitive Implications

8.1 Beyond the Single Lineage

The preceding chapters treat self-improvement as a property of a single computational lineage. The operator-ecology framework introduced in Section 1.5, however, was never confined to single systems: it was developed to explain the persistence of distinctions, practices, and institutions generally, of which a self-modifying program is one instance among many. Read at that scale, the media quine is not a novel computational device but a formal name for something civilizations have always depended on — an archive, a curriculum, an apprenticeship, a scientific tradition — namely an artifact whose successful transmission installs, in the next generation of practitioners, the operator capable of producing further work of the same kind, not merely a record of past work. What the preceding chapters add to this older picture is a criterion for when such transmission is actually succeeding (Definition 3.1) and a diagnosis of the specific way it can fail to succeed while still looking, from the outside, like healthy continuity (Sections 7.2–7.3).

8.2 Keystone Operators Revisited

The operator-ecology account distinguishes keystone operators — those that support many other operators without necessarily being frequently exercised themselves, such as literacy, apprenticeship, or archival practice — from operators whose loss is locally costly but does not cascade. The media-quine criterion sharpens what it means for a cultural artifact to function as a keystone operator: it is an artifact that, when correctly interpreted, does not merely transmit its own content but transmits the capacity to produce and maintain other artifacts in its lineage, including, crucially, the capacity to recognize and repair future artifacts that fail the same criterion. A civilization’s most consequential documents, on this view, are not its largest archives but its media quines: the comparatively small set of texts, practices, or institutions whose correct transmission regenerates the operators that keep the rest of the archive legible.

8.3 Cognition as a Self-Modification Lineage

Nothing in Chapters 5–7 is specific to software. An individual mind, understood as a system that continually revises its own dispositions, habits, and beliefs in light of experience, is a self-modification lineage in exactly the sense of Definition 5.1: each state of the mind is produced from the previous one by an internal operator, and the question of whether a given episode of learning constitutes genuine understanding, as opposed to a drift in behavior that merely looks like understanding from the outside, is the cognitive instance of the drift/reconstruction distinction from Section 5.3. A learner who can only reproduce an answer has received an ordinary document, in the vocabulary of Section 3.3; a learner who can generate new instances of the same kind of reasoning without further instruction has received a media quine. This reframes a familiar pedagogical distinction — between rote and generative understanding — as a special case of the formal criterion developed for computational self-improvement, and suggests that the failure modes catalogued in Chapter 7, particularly the silent loss of the Chain-of-Memory fixed point described in Section 7.3, have direct analogues in education: a curriculum can remain formally complete, in the sense of covering all the required material, while ceasing to be generatively transmissible, in the sense that matters.

8.4 Media Quines Across Domains

It is worth closing the substantive argument with a few concise instances of Definition 3.1 outside the computational setting that has occupied most of this monograph, not to survey the space exhaustively but to indicate its breadth. In mathematics, a proof that teaches proof construction — one whose exposition makes the discovery process legible, not merely the validated result — is a media quine with respect to mathematical practice, in contrast to a proof that states only what is true and how to verify it, which transmits a result without transmitting the operator that found it. In programming, a codebase that teaches architectural reasoning, through consistent structure and legible design decisions, functions as a media quine with respect to software design, in contrast to a codebase that exposes only its API surface, which transmits usage without transmitting the judgment that shaped the design. In science, a paper that generates new researchers — one whose method section is detailed and motivated enough that a reader can extend the research program, not merely cite its conclusions — is a media quine with respect to a research tradition, in contrast to a paper that reports results without transmitting the experimental judgment that produced them. In education, a curriculum that produces independent teachers, capable of designing new instruction rather than only delivering the instruction they were given, is a media quine with respect to pedagogy itself, in contrast to a curriculum that produces competent students who can pass the curriculum’s own assessments but cannot construct new ones. And at the largest scale, an institution that regenerates the practices needed to maintain itself — training its own successors in the judgment, not merely the procedures, of institutional maintenance — is a media quine with respect to its own persistence, in exactly the sense the two-cities contrast of the operator-ecology account was built to illustrate. What unites these five cases is the same distinction that has organized the whole monograph: in each, the object of interest is not what the artifact says or does at a given moment, but whether engaging with it regenerates, in the next practitioner, the operator capable of producing more of the same kind.

8.5 Closing Remark

The argument of this monograph has been that “self-improving” is not a property a system either has or lacks by virtue of modifying its own code, but a property that must be engineered into what a system retains across modification, and that the right unit for engineering that retention is not the individual state but the artifact-as-operator-factory formalized here as a media quine. Whether the specific architecture sketched in Chapter 6 is the most practical way to satisfy that requirement is a separate, open question; the more durable claim is the one made in Chapters 3 and 5, that any architecture aiming at genuine self-improvement, computational or otherwise, will have to answer the question Definition 3.1 makes precise: not merely what a system produces at each step, but whether what it produces is capable of regenerating the operator that produced it.

Appendix A

Admissibility Spaces and Continuation Geometry

This appendix states, in a single self-contained place, the minimal geometric machinery underlying Chapters 1 and 2. Nothing here is required to follow the main text, which uses these notions informally; it is provided for readers who want the formal skeleton made explicit.

Definition A.1 (Admissibility Space). *An admissibility space is a pair (X, A) where X is a set of states and $A \subseteq X \times X$ is the admissible transition relation: $(x, y) \in A$ means the system can move from x to y by a single admissible operator.*

Definition A.2 (Admissible Path). *An admissible path from x to y of length n is a sequence $x = x_0, x_1, \dots, x_n = y$ with $(x_i, x_{i+1}) \in A$ for each i . The length-0 path from x to x is always admissible.*

Definition A.3 (Admissibility Volume). *The admissibility volume of x is $V(x) = \{y \in X : \text{there exists an admissible path from } x \text{ to } y\}$.*

Proposition A.4 (Composition of Admissible Paths). *If there is an admissible path from x to y and an admissible path from y to z , there is an admissible path from x to z .*

Proof. Concatenate the two sequences at their shared endpoint y ; the result satisfies the admissible-path condition at every step, including the join, since the two constituent paths already do. $\square$

Proposition A.5 (Admissibility as a Preorder). *Define $x \preceq y$ iff $y \in V(x)$. Then $\preceq$ is a preorder on X : reflexive (by the length-0 path) and transitive (by the composition proposition above).*

Definition A.6 (Continuation Class). *Define $x \sim y$ iff $x \preceq y$ and $y \preceq x$ (mutual reachability). This is an equivalence relation, being the symmetric kernel of a preorder. The equivalence class $[x]_\sim$ is the continuation class of x .*

Proposition A.7 (Partition and Induced Order). *The continuation classes partition X , and $\preceq$ descends to a partial order on the quotient $X/\sim$.*

Proof. Partition: immediate from $\sim$ being an equivalence relation. Induced order: $\preceq$ is well-defined on classes because if $x \sim x'$ and $x \preceq y$ then $x' \preceq y$ by transitivity through x ; antisymmetry on the quotient holds by construction, since $[x]_\sim \preceq [y]_\sim$ and $[y]_\sim \preceq [x]_\sim$ together mean $x \sim y$, i.e. $[x]_\sim = [y]_\sim$. $\square$

Read against the main text, X is the space of possible system states, A is the set of single-step operators available to a self-modifying system (Bind–Collapse compositions, in the vocabulary of Chapter 6), and $V(x)$ is the admissibility volume invoked informally throughout Chapter 1. A self-modification lineage $M_0, M_1, \dots$ (Section 5.2) is precisely an admissible path in this sense, and the requirement that continuation-preserving recursion (Section 2.3) keep the lineage inside a single continuation class is the requirement that every M_{i+1} satisfy $M_i \sim M_{i+1}$ rather than merely $M_i \preceq M_{i+1}$: forward reachability alone permits one-way drift into an unrecoverable region of X , while remaining in a shared continuation class additionally guarantees a return path.

Appendix B

Media Quines as Fixed Points

Definition B.1 (Interpretation and Production). Let $\mathcal{M}$ be a space of media objects and $\mathcal{G}$ a space of generative operators. An interpretation operator is a map $I : \mathcal{M} \rightarrow \mathcal{G}$; a production operator is a map $P : \mathcal{G} \rightarrow \mathcal{M}$ with $P(G)$ denoting a (possibly non-unique) artifact produced by G .

Definition B.2 (Exact Media Quine). $G \in \mathcal{G}$ is an exact media quine if $I(P(G)) = G$, i.e. G is an exact fixed point of $I \circ P : \mathcal{G} \rightarrow \mathcal{G}$.

The exact case is almost never achieved in practice, which is why Definition 3.1 in the main text is stated up to admissible equivalence rather than exact identity. Formalizing that relaxation:

Definition B.3 (Admissible Equivalence on $\mathcal{G}$). Let $\approx$ be an equivalence relation on $\mathcal{G}$ such that $G \approx G'$ implies $V(G) = V(G')$ in the sense of Appendix A (equivalent operators generate the same admissibility volume of possible outputs). G is a media quine (in the sense of Definition 3.1) if $I(P(G)) \approx G$.

Proposition B.4 (Iterative Reconstruction). Define the sequence $G_0 \in \mathcal{G}$, $G_{n+1} = I(P(G_n))$. If $G_{N+1} \approx G_N$ for some N , then G_N is a media quine.

Proof. Immediate from the definition: $G_{N+1} = I(P(G_N)) \approx G_N$ is exactly the media-quine condition applied to G_N . $\square$

This iterative formulation is the formal counterpart of the reading-as-execution picture of Section 4.2: repeated interpretation-then-production is a concrete process by which a candidate operator can be tested for the fixed-point property, rather than merely asserted to have it, and a lineage that fails to stabilize under this iteration — one for which $G_{n+1} \not\approx G_n$ for all n up to some resource bound — is a lineage exhibiting drift in the sense of Section 5.3 at the level of a single artifact’s interpretation rather than across generations.

Proposition B.5 (Stability Under Admissible Perturbation). Suppose $I \circ P$ respects $\approx$, i.e. $G_1 \approx G_2 \implies I(P(G_1)) \approx I(P(G_2))$. If G is a media quine and $G' \approx G$, then G' is also a media quine.

Proof. $I(P(G')) \approx I(P(G))$ by the respecting hypothesis, and $I(P(G)) \approx G \approx G'$ by assumption; the media-quine property is preserved by transitivity of $\approx$. $\square$

The respecting hypothesis is not automatic; it is precisely the condition Section 7.2 shows can fail under gradual, compositionally admissible drift of the interpretation operator itself, which is why Proposition B.3 is stated conditionally rather than as an unconditional guarantee.

Appendix C

Reconstruction Metrics

Definition C.1 (Reconstruction Content). *Given a decomposition of a generative operator $G = G_1 \circ \dots \circ G_k$ into components (as in the partial-media-quine construction of Section 3.4), define $R(M_i, M_j) \in [0, 1]$ as the fraction of components G_ℓ admissibly recoverable from M_j using M_i as a reference: $R(M_i, M_j) = |\{\ell : \widehat{G}_\ell \text{ recoverable, } \widehat{G}_\ell \approx G_\ell\}|/k$.*

Proposition C.2 (Non-Monotonicity of R Along a Lineage). *For a self-modification lineage $M_0, M_1, \dots$, $R(M_0, M_i)$ need not be monotonically decreasing in i .*

Remark C.3. *This is a deliberate departure from what one might expect of a degradation measure, and it is a feature rather than a defect of the definition: a later generation can repair components lost at an intermediate generation (Section 5.3’s repairable case), recovering operator content that a naive monotonicity assumption would treat as permanently lost. R tracks recoverable content at a given pair of generations, not a cumulative or irreversible decay curve.*

Remark C.4 (Failure of the Triangle Inequality). *R is not a metric in the technical sense, and in particular need not satisfy anything resembling a triangle inequality: $R(M_0, M_2)$ can be strictly less than what any combination of $R(M_0, M_1)$ and $R(M_1, M_2)$ would suggest, because reconstructing G through an intermediate generation can lose information that a direct comparison would not, and conversely $R(M_0, M_2)$ can exceed naive expectations if M_2 independently re-derives structure that M_1 had discarded. R should be read as a directional, context-dependent recoverability score, not as a distance.*

Definition C.5 (Repair Distance and Admissibility Radius). *The repair distance $d_{\text{rep}}(x, y)$ is the minimum length of an admissible path (Appendix A) connecting x and y within a shared continuation class, or ∞ if none exists. The admissibility radius of a continuation class $[x]_{\sim}$ is $\sup_{y \in [x]_{\sim}} d_{\text{rep}}(x, y)$: the greatest repair distance any member of the class is from x , a measure of how much accumulated modification the class can absorb before an external repair process would need unboundedly many steps.*

These metrics connect the reconstruction/repair cluster’s existing diagnostic-coordinate machinery to the present monograph’s lineage vocabulary: R is the artifact-level counterpart of what Diagnostic Coordinates and the Time of Repair treats at the level of general degraded systems, specialized here to the media-quine setting.

Appendix D

Category-Theoretic Formulation

This appendix sketches, rather than fully develops, a categorical reading of the preceding machinery, offered as a direction for future formalization.

Definition D.1 (The Category **Gen**). *Let **Gen** be the category whose objects are generations $M_i = (C_i, H_i)$ (Section 6.2) and whose morphisms $M_i \rightarrow M_j$ are admissible reconstructions: pairs $(\hat{G}, \pi)$ where $\hat{G}$ is an operator admissibly equivalent to some G with $G(M_i) = M_j$, together with the evidence π (a Spharepop history suffix, in the sense of Section 6.2) witnessing that $\hat{G}$ is recoverable from M_j .*

Composition of morphisms is given by composing the underlying operators and concatenating the witnessing history suffixes; the identity morphism on M_i is the length-0 admissible path of Appendix A, which trivially reconstructs the identity operator from M_i itself. Associativity and the identity laws follow from the corresponding properties of operator composition and history concatenation.

Definition D.2 (Reconstructible Lineage as a Functor). *Let $\mathbb{N}$ denote the poset category of natural numbers under $\leq$. A self-modification lineage is reconstructible (Definition 5.1) precisely when the functor $F : \mathbb{N} \rightarrow \mathbf{Gen}$ sending $i \mapsto M_i$ and $(i \leq i+1) \mapsto (\hat{G}_i, \pi_i)$ is well-defined on every generating morphism, i.e. every successor morphism in the diagram is realized by an actual object of **Gen** rather than merely asserted.*

Proposition D.3 (Global Reconstructibility as Functoriality). *A lineage is globally reconstructible (Section 5.6) iff F preserves composition along the full chain $0 \rightarrow 1 \rightarrow \dots \rightarrow n$ for every n , i.e. the composite morphism $F(0 \rightarrow n)$ computed via the functor agrees, up to $\approx$, with direct composition of the underlying operators $G_{n-1} \circ \dots \circ G_0$.*

This restates Section 5.6's local/global distinction in categorical language: local reconstructibility is functoriality on generating morphisms (adjacent steps), while global reconstructibility additionally requires that functoriality compose correctly over arbitrarily long chains, which is not automatic in a category where morphism composition is only defined up to $\approx$ rather than exactly.

Remark D.4 (Chain of Memory as a Universal Property). *Chain of Memory (Section 1.4) admits a natural reading as a limit condition rather than an algorithmic criterion: a state M is a Chain-of-Memory fixed point with respect to I if the cone formed by M together with the morphism $I(M) \rightarrow M$ (production from the recovered operator) is terminal among cones satisfying the reconstruction condition, in the sense*

that any other candidate reconstruction factors through it. A full categorical development of this remark — specifying the relevant diagram and verifying the universal property — is left for future work; it is offered here as the appropriate target for such a development rather than as an established result.

Appendix E

Spherepop Algebra

The main text (Section 4.2) introduces Pop, Refuse, Bind, and Collapse informally, as an alphabet rather than a fully specified calculus, deferring the algebraic development to the existing Spherepop corpus, where the functional-completeness result invoked in Section 4.6 is established directly. This appendix collects the signatures and composition laws needed to read Chapters 4 and 6 algebraically, without re-deriving completeness here.

Definition E.1 (Primitive Signatures). *Over a history H and content C : $\text{Pop} : H \rightarrow H_{\text{sel}}$ selects a sub-history (selection); $\text{Refuse} : H_{\text{sel}} \rightarrow H_{\text{sel}}$ excludes elements failing an admissibility predicate (exclusion); $\text{Bind} : C \times \delta \rightarrow H_{\text{ev}}$ couples content to a candidate event, appending a coupling record (coupling); $\text{Collapse} : H_{\text{ev}} \rightarrow C$ projects a coupled event down to new content (projection).*

Definition E.2 (Composition and Normal Form). *Any composite operator built from Pop, Refuse, Bind, Collapse under sequential composition $\circ$ is in normal form if no two adjacent primitives of the same type appear without an intervening primitive of a different type, since $\text{Refuse} \circ \text{Refuse} = \text{Refuse}$ and $\text{Pop} \circ \text{Pop} = \text{Pop}$ on a fixed history (both are idempotent selection/exclusion operators over the same predicate).*

Proposition E.3 (The Verification and Modification Operators as Normal Forms). *The verification operator of Section 6.4, $I = \text{Refuse} \circ \text{Pop}$, and the modification operator of Section 6.3, $G = \text{Collapse} \circ \text{Bind}$, are each already in normal form, being minimal two-primitive compositions drawing on disjoint primitive pairs.*

Remark E.4 (Functional Completeness). *The claim that Sphere, Merge, Choice, Link, Unlink, Set-Meta, and Nest all reduce to compositions of these four primitives is proved in the Spherepop calculus’s Operator Representation Theorem, established independently of the present monograph. This appendix does not reprove that theorem; it records only the two composite forms (I and G) actually used in Chapter 6’s architecture, since those are the composites on which Proposition 5.2 depends.*

Appendix F

Dynamical Systems

Definition F.1 (Recursive Update). *A self-modification lineage is a discrete dynamical system $M_{n+1} = G_n(M_n)$ on the state space X of Appendix A, where the update rule G_n may itself vary with n (a non-autonomous system, reflecting that the modification operator is not fixed in advance but is produced anew, possibly by a different process, at each step).*

Remark F.2 (Fixed Points and Media Quines). *An exact fixed point of the update rule, $G(M^*) = M^*$, is the dynamical-systems counterpart of the degenerate classical-quine case of Section 3.1: a lineage that has stopped changing. This is a substantially weaker notion than a media quine in the sense of Definition 3.1, which requires only that successive states remain reconstructible, not that they coincide; a genuinely self-improving lineage is expected to have no fixed points at all in this dynamical sense while still satisfying the Continuation Criterion at every step.*

Remark F.3 (Drift as Divergence, Repair as a Basin of Attraction). *Drift (Section 5.3) corresponds, in this vocabulary, to a trajectory that leaves the admissibility volume $V(M_0)$ without bound, analogous to divergence from an attracting region. A repairable lineage corresponds to a trajectory that remains within some basin of attraction around the origin's continuation class even after individual steps that would, uncorrected, have carried it out — the external scaffolding of Section 5.3 plays the role of a restoring force. Reconstructibility in the strong sense of Definition 5.1 corresponds to the trajectory itself carrying, at each point, the information needed to compute the restoring force without external intervention.*

Remark F.4 (Trust Boundaries as Bifurcation Structure). *The trust boundary of Section 6.6 — the fixed Pop/Refuse layer beneath the modifiable content C_i — is structurally analogous to a bifurcation parameter held constant while the state variable is allowed to evolve: were the boundary itself subject to the dynamics, the qualitative structure of the admissible region (which points count as reachable, which modifications as accepted) could itself bifurcate, precisely the gradual-catastrophe scenario of Section 7.3 read as a slow drift of an otherwise-fixed parameter into a regime where the system's qualitative behavior changes discontinuously.*

Appendix G

Information Without Representation

Definition G.1 (Operator-Theoretic Information Content). *For a media object M with interpretation operator I , define the information content of M as*

$$\mathcal{I}(M) = [G]_{\approx}, \quad G = I(M),$$

the admissible equivalence class (Appendix B) of the generative operator recovered by interpreting M , rather than a quantity measured over a probability distribution on symbols.

Remark G.2 (Contrast with Shannon Information). *Shannon information is defined over a source’s symbol distribution and quantifies surprise or channel capacity; it is agnostic to what the symbols mean or what operator produced them, and two artifacts with identical Shannon entropy can have $\mathcal{I}(M)$ values that are entirely unrelated equivalence classes, if their interpretation operators recover unrelated generative processes. Conversely, two artifacts with very different Shannon entropy — one terse, one verbose — can have the same $\mathcal{I}(M)$, if both admit interpretation to admissibly equivalent operators. $\mathcal{I}$ and Shannon entropy are answering different questions: Shannon information asks how surprising a message is against a background distribution; $\mathcal{I}(M)$ asks what generative capacity successfully interpreting M installs in the interpreter.*

Remark G.3 (Consequence for Compression). *Section 3.6’s discussion of compression follows directly from this definition: a compression scheme preserves information in the sense of $\mathcal{I}$ exactly when it preserves the equivalence class $[G]_{\approx}$, independent of how much it reduces the entropy or bit-length of M itself. This is a strictly different optimization target from standard rate-distortion theory, which optimizes bit-length against a fidelity measure on M ’s surface content; optimizing $\mathcal{I}$ -preservation instead optimizes bit-length against fidelity to M ’s generative operator, and the two targets can recommend different compressed representations for the same source.*

This appendix’s definition is offered as the formal cash value of the monograph’s recurring claim that generative capacity, not stored content, is the right unit of informational value for artifacts intended to function as media quines; it is not proposed as a replacement for Shannon information in domains — channel coding, storage engineering — where the classical theory’s actual target quantity is what is wanted.

Bibliography

Self-reference and computation

- [1] W. V. O. Quine, *The Ways of Paradox and Other Essays*, Harvard University Press, 1976.
- [2] D. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, Basic Books, 1979.
- [3] D. Hofstadter, *I Am a Strange Loop*, Basic Books, 2007.
- [4] S. C. Kleene, "On notation for ordinal numbers," *Journal of Symbolic Logic*, 1938.
- [5] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, 1936.
- [6] J. von Neumann, *Theory of Self-Reproducing Automata*, edited by A. Burks, University of Illinois Press, 1966.
- [7] G. Chaitin, *Algorithmic Information Theory*, Cambridge University Press, 1987.

Programming languages and self-modification

- [8] H. Abelson, G. J. Sussman, J. Sussman, *Structure and Interpretation of Computer Programs*, MIT Press, 1985.
- [9] J. Backus, "Can programming be liberated from the von Neumann style?," *Communications of the ACM*, 1978.
- [10] L. Lamport, *Specifying Systems: The TLA+ Language and Tools*, Addison-Wesley, 2002.
- [11] D. Harel, "Statecharts: A visual formalism for complex systems," *Science of Computer Programming*, 1987.
- [12] E. W. Dijkstra, *Selected Writings on Computing: A Personal Perspective*, Springer, 1982.

Information and communication

- [13] C. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, 1948.
- [14] C. Shannon, W. Weaver, *The Mathematical Theory of Communication*, University of Illinois Press, 1949.
- [15] G. Bateson, *Steps to an Ecology of Mind*, University of Chicago Press, 1972.

- [16] L. Floridi, *The Philosophy of Information*, Oxford University Press, 2011.
- [17] F. Dretske, *Knowledge and the Flow of Information*, MIT Press, 1981.

Cybernetics and systems

- [18] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, MIT Press, 1948.
- [19] W. R. Ashby, *An Introduction to Cybernetics*, Chapman & Hall, 1956.
- [20] S. Beer, *Brain of the Firm*, Wiley, 1972.
- [21] H. Maturana, F. Varela, *Autopoiesis and Cognition: The Realization of the Living*, Reidel, 1980.
- [22] F. Varela, E. Thompson, E. Rosch, *The Embodied Mind*, MIT Press, 1991.

Repair, persistence, and memory

- [23] D. Hume, *A Treatise of Human Nature*, 1739.
- [24] D. Parfit, *Reasons and Persons*, Oxford University Press, 1984.
- [25] A. N. Whitehead, *Process and Reality*, Macmillan, 1929.
- [26] G. Simondon, *L'individuation à la lumière des notions de forme et d'information*, Millon, 1964.
- [27] N. Goodman, *Ways of Worldmaking*, Hackett, 1978.
- [28] A. Juarrero, *Dynamics in Action: Intentional Behavior as a Complex System*, MIT Press, 1999.

Archives and external cognition

- [29] N. Luhmann, *Social Systems*, Stanford University Press, 1995.
- [30] V. Bush, "As we may think," *The Atlantic*, 1945.
- [31] T. Nelson, *Literary Machines*, Mindful Press, 1981.
- [32] D. Engelbart, "Augmenting human intellect: A conceptual framework," SRI Summary Report, 1962.
- [33] A. Clark, D. Chalmers, "The extended mind," *Analysis*, 1998.

Category theory and algebra

- [34] S. Mac Lane, *Categories for the Working Mathematician*, Springer, 1971.
- [35] E. Riehl, *Category Theory in Context*, Dover, 2016.
- [36] F. W. Lawvere, S. Schanuel, *Conceptual Mathematics: A First Introduction to Categories*, Cambridge University Press, 1997.
- [37] B. Jacobs, *Introduction to Coalgebra: Towards Mathematics of States and Observation*, Cambridge University Press, 2016.

Dynamical systems

- [38] V. I. Arnold, *Ordinary Differential Equations*, MIT Press, 1973.
- [39] S. Smale, "Differentiable dynamical systems," *Bulletin of the AMS*, 1967.
- [40] R. Thom, *Structural Stability and Morphogenesis*, Benjamin, 1972.
- [41] I. Prigogine, I. Stengers, *Order Out of Chaos*, Bantam, 1984.

Machine learning and self-improvement

- [42] R. Sutton, A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [43] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2020.
- [44] J. Pearl, *Causality: Models, Reasoning, and Inference*, Cambridge University Press, 2000.
- [45] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, 2017.
- [46] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.

Version control and software evolution

- [47] K. Fogel, *Producing Open Source Software*, O'Reilly, 2005.
- [48] J. Hamano et al., *Git*, <https://git-scm.com>, source-control software, ongoing.
- [49] T. Mens, S. Demeyer (eds.), *Software Evolution*, Springer, 2008.